

PROGRAMACIÓN PARA LA CIENCIA DE DATOS · SCY1101

Bank Marketing Analytics

Solución end-to-end: pipeline ETL multifuente, API y dashboard diferenciado por audiencia

Evaluación Parcial N°3 · Defensa individual (15 min)

Equipo: Benjamin Arratia · Angel Palominos · María José Álvarez

Repositorio: github.com/PalominosAngel/bank-marketing-analytics_EV3

Stack: Python 3.12 · pandas · scikit-learn · FastAPI · Streamlit · Plotly

El problema: campañas que se deciden sin datos

- Un banco ejecuta campañas telefónicas para captar depósitos a plazo, pero **no sabe qué perfiles ni qué canales convierten**.
- **Objetivo:** transformar datos crudos de campañas en información accionable para tres audiencias distintas — **gerencia, analistas y personal operativo**.
- Punto de partida real: **4.521 contactos** con solo **11,5% de conversión** → hay margen claro para focalizar el esfuerzo comercial.

Valor: pasar de “llamar a todos” a “llamar a quien más probabilidad de conversión tiene”, con evidencia y no por intuición.

Una plataforma end-to-end, no un script suelto

1 Integración

- 3 fuentes heterogéneas
- CSV + API REST + SQL
- Validación de esquema

2 Procesamiento

- Pipeline ETL robusto
- Segmentación PCA + K-Means
- Manejo avanzado de errores

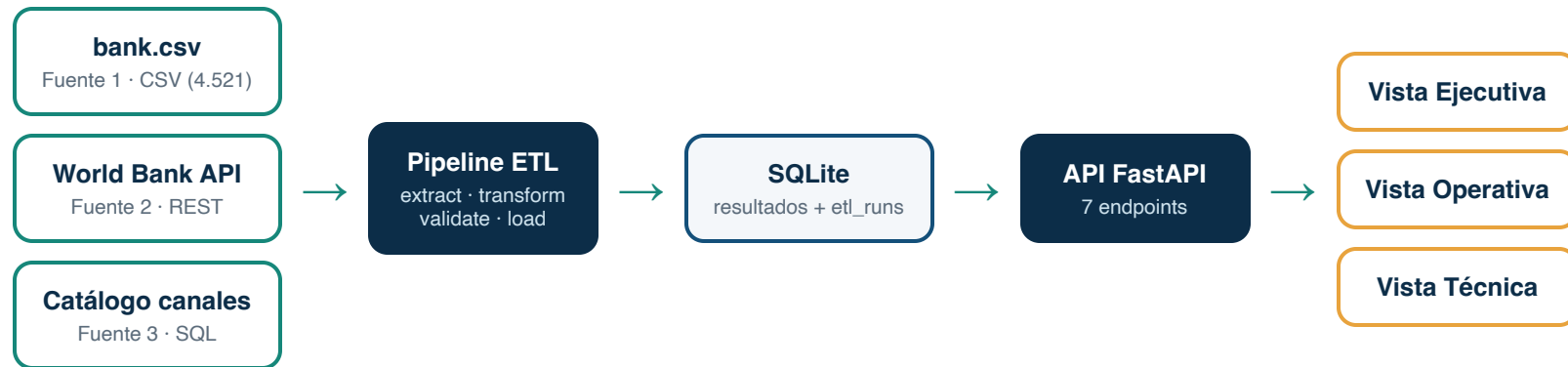
3 Servicio

- API REST (FastAPI)
- Desacopla datos de UI
- 7 endpoints documentados

4 Consumo

- Dashboard por audiencia
- Ejecutiva / Operativa / Técnica
- Ejecución local reproducible

Flujo de datos en capas desacopladas



Decisión clave: el dashboard nunca toca la base directamente — consume la API. Esto permite cambiar el almacenamiento o escalar sin tocar la visualización.

Robustez: integra, valida y tolera fallos

- **3 fuentes reales** orquestadas: CSV, API REST del World Bank (Portugal, 2018–2024) y tabla SQL de referencia.
- **Validación de esquema:** columnas requeridas, tipos, dominios binarios, nulos y outliers.
- **Trazabilidad:** cada corrida se registra en **etl_runs** con estado y métricas de calidad.

Manejo avanzado de errores

- Reintentos con **backoff exponencial** + timeout en la API externa
- **Fallback a caché** local si la red falla
- Transacciones con **rollback** en la carga
- Logging profesional con rotación de archivos

Decisiones técnicas y por qué

Decisión	Alternativa	Por qué la elegimos
SQLite (con soporte Postgres vía DATABASE_URL)	Postgres fijo	Baja fricción de instalación; portable sin perder la opción de escalar
FastAPI intermedia	Dashboard lee la BD directo	Desacopla datos de visualización; valida y documenta (OpenAPI)
Caché de la API externa	Llamada directa siempre	Tolerancia a fallos de red; demo estable y reproducible
PCA + K-Means descriptivo	Modelo predictivo	Segmentación exploratoria honesta; no afirma causalidad
Ejecución local + .env	Infraestructura dedicada	Configuración externa y demo reproducible sin dependencias de infraestructura

Para gerencia: el negocio en una mirada

4.521

Clientes contactados

11,52%

Tasa de conversión

521

Conversiones

1.423

Saldo promedio (€)

- Lenguaje de **negocio**: conversión, perfiles con mejor respuesta, contexto macroeconómico.
- Responde la pregunta del gerente: **¿en quién enfoco la próxima campaña?**

Para el equipo de campañas: accionable

- **Filtros interactivos** por ocupación, segmento y solo-conversiones.
- Rendimiento **por canal de contacto** y duración de llamadas por resultado.
- **Descarga CSV** de la muestra filtrada para trabajar la lista de llamado.

Valor operativo

- Pasa del “qué pasó” al “**a quién llamo hoy**”
- Prioriza canales y franjas más efectivas
- Reduce esfuerzo en contactos de baja probabilidad

Para analistas: trazabilidad y modelo

Calidad de datos

- Métricas de validación
- Nulos y outliers
- Severidad por regla

Trazabilidad ETL

- Historial de etl_runs
- Estado y filas por corrida

Segmentación

- Resumen K-Means
- Dispersión PCA
- Matriz de correlación

Diferenciación real: la misma data, tres lenguajes distintos. La vista técnica habla de PCA y correlación; la ejecutiva, de conversión y saldo.

El pipeline funcionando, de punta a punta

Paso	Qué se muestra	Resultado real
1. ETL	<code>python -m etl.pipeline</code> · 3 fuentes	4.521 filas · estado <code>live_api</code>
2. API	<code>uvicorn api.main:app</code> · Swagger /docs	200 · conversión 11,52%
3. Dashboard	<code>streamlit run</code> · 3 vistas por audiencia	render Plotly en vivo

Plan B: si la demo en vivo falla por red, las capturas y los KPIs reales respaldan que el flujo corre. Nunca depender solo de la conexión del laboratorio.

Git como evidencia de trabajo en equipo

- Flujo de ramas: **main** → **develop** → **feature/***.
- **Issues** con responsables y criterios de aceptación.
- **Pull Requests** con revisión cruzada antes del merge.
- Convención de commits (**feat/fix/docs/test**).

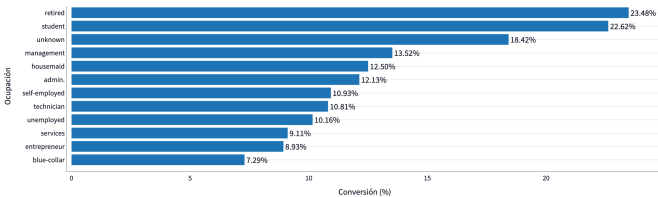
Metodología

- Reparto de responsabilidades por componente
- Tablero de issues como gestión del trabajo
- Revisión de pares = calidad y conocimiento compartido
- main reservada para versiones estables

Los gráficos reales, uno por audiencia

EJECUTIVA

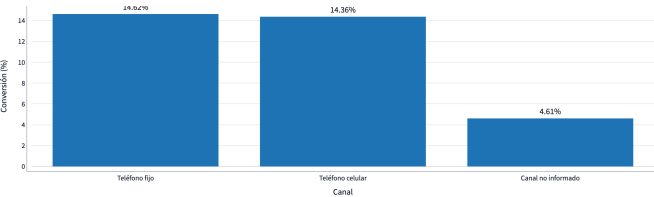
Tasa de conversión por ocupación: perfiles con mejor respuesta



Conversión por ocupación. Retired y student lideran → dónde enfocar la campaña.

OPERATIVA

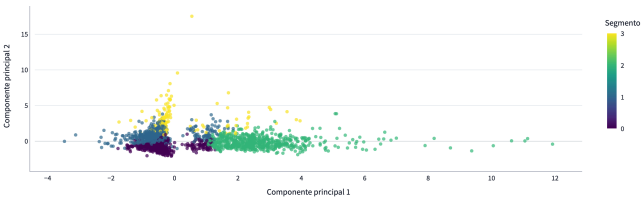
Rendimiento por canal de contacto



Rendimiento por canal. El canal informado convierte 3x más que el no informado.

TÉCNICA

PCA en dos dimensiones para visualizar segmentos K-Means



Segmentación PCA + K-Means. Cuatro segmentos de clientes proyectados en 2D.

Capturas tomadas directamente del dashboard en ejecución — no son mockups.

Testing automatizado

9 / 9

Tests pasando

5

Módulos cubiertos

7

Endpoints API

- Pruebas significativas: validación fail-fast, **fallback de red** de la API, generación de columnas de negocio y clustering.
- Ejecución reproducible con `pytest`; evidencia guardada como informe de pruebas.

Lecciones aprendidas y mejoras

Qué funcionó

- La **separación en capas** facilitó testing y cambios
- El manejo de errores robusto evitó demos frágiles
- El caché dio resiliencia ante fallos de red

Oportunidades de mejora

- Optimizar el ETL para **grandes volúmenes** (chunking, batch)
- Caché en el dashboard para mejor rendimiento
- Autenticación en la API y scheduling del ETL
- Monitoreo y despliegue en la nube

GRACIAS

De datos crudos a decisiones

Pipeline ETL multifuente · API · dashboard por audiencia · testing · Git profesional

Repositorio: github.com/PalominosAngel/bank-marketing-analytics_EV3

Demo local: `python -m etl.pipeline → uvicorn api.main:app → streamlit run dashboards/app.py`

¿Preguntas?